ROS 2: advanced concepts

Dawid Seredyński

Warsaw University of Technology

June 2026

Aim of this lecture

To introduce more advanced concepts that allow to design and run *ROS 2* systems

Learning goals

By the end of this module, you should be able to:

- create a properly–defined launch file
- create a node and configure it
- create a interface

Launching

Motivation:

- lot of programs, *node*s
- names, *namespace*s, remappings
- configurations, *parameter*s
- order of execution, events, waiting
- reusability, parametrization, scalability

Running manually gets cumbersome quite quickly

Running a node: command line, launch.xml

```
ros2 run my_robot_pkg camera_node \
  --input /dev/video0 \
  --ros-args \
  -r __ns:=/robot1 \
  -r __node:=front_camera \
  -r image_raw:=camera/image_raw \
  -p use_sim_time:=true \
  -- \
  --verbose --save-debug-images
```

```
<launch>
  <node pkg="my_robot_pkg" exec="camera_node"
        namespace="/robot1"
        name="front_camera"
        args="--input /dev/video0 --verbose --save-debug-images"
        output="screen">
    <remap from="image_raw" to="camera/image_raw"/>
    <param name="use_sim_time" value="true"/>
  </node>
</launch>
```

Launch system

The *launch system*★:

- allows to describe the configuration of system and to execute it as described
- provides means to specify:
  - what to run
  - where to run
  - what arguments / *parameter*s to pass
  - ROS-specific conventions (*namespace*s, remappings)
  - how to monitor the state of the launched processes
  - how to react to changes of the state
- is written in Python
- is a part of *ROS 2* core

Launch file

A *launch file*★ can:

- have arguments
- include other *launch file*s
- start a *node*, and:
  - set arguments / *parameter*s
  - set name, set *namespace*, remap *topic*s
- start an executable / command
- run a command periodically
- register event handlers, emit events
- evaluate values of expressions

A *launch file*:

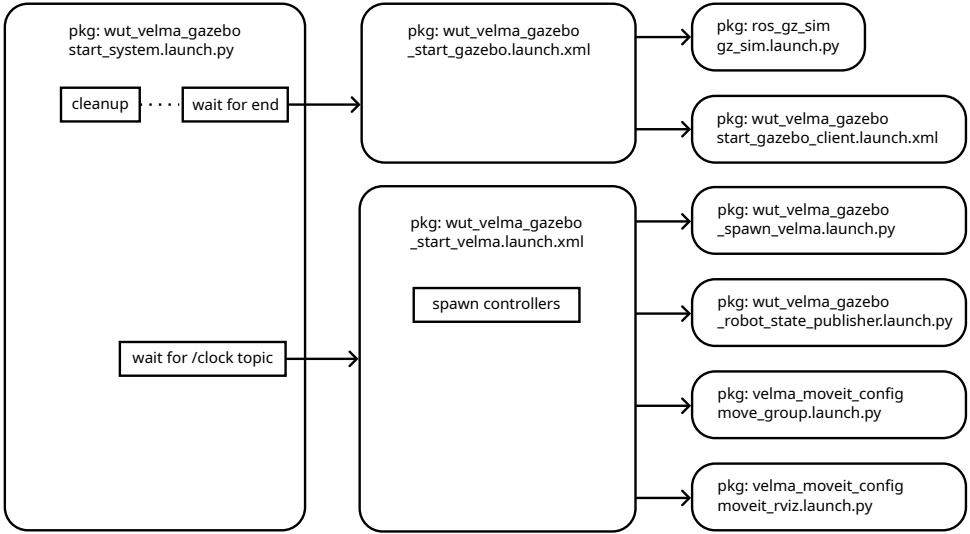
- generates a **launch description**
- can be written in `xml`, `yaml`, Python
- can include other *launch file*s: mixing .xml, .yaml and .py is possible

Recommended design patterns for *launch file*s written in:

- xml, yaml: good for static structure
  - simple, clear
  - less expressive: no events, no order, no scripting
- Python: good for bring-up, events, order, orchestration
  - more expressive
  - more complex

Launching a system

- A user starts a *launch file*, e.g.:
  `ros2 launch my_package run_system.launch.py`
- The *launch system* loads a launch description from the file.
  This description contains launch actions:
  - start *node*s
  - include other *launch file*s
  - declare arguments
  - set *parameter*s
  - register event handlers, etc.
- The *launch system* executes the description:
  - evaluates substitutions
  - starts processes
  - expands includes
  - reacts to events



Node-to-*node* connections are established via **discovery**

*Node*s do not require a central broker to start communicating

- however, some middleware implementations allow discovery server★

Discovery★ – *node*s automatically find each other and learn what *topic*s, *service*s, *action*s, and *QoS* settings exist

- when a *node* starts, it advertises its presence to other nodes on the same ROS domain
- other *node*s respond with information about themselves
- the graph becomes connected

Domain

*Node*s only discover other *node*s in the same ROS domain

- ROS domain is identified by `ROS_DOMAIN_ID`★ (an environment variable)
- `ROS_DOMAIN_ID` is used by DDS to compute UDP ports used for discovery and communication
- up to 120 participants (*node*s) in one domain
- up to about 120 `ROS_DOMAIN_ID`s

Multiple *ROS 2* systems may run in one network without interference

Controlling how far *ROS 2* automatic discovery is allowed to reach:

- `ROS_AUTOMATIC_DISCOVERY_RANGE` – an environment variable

Node parameters

A *parameter* is a named value stored by a *node*

- it is used to configure the *node* at startup or during runtime
- *parameter*s are associated with individual *node*s

A parameter has:

- name
- value with a specified type
- descriptor

A *node* declares its *parameter*s at startup

Not recommended: dynamic typing (changing type at run–time)

Setting parameters

```
ros2 run my_pkg my_node --ros-args -p max_velocity:=0.5
```

```
ros2 run my_pkg my_node --ros-args --params-file config/par.yaml
```

YAML structure:

```
node_name:
  ros__parameters:
    parameter_name:  value
```

```
ros2 param set /my_node max_velocity 0.8
```

Also:

- in a launch file
- by calling a service `/node_name/set_parameters` or `/node_name/set_parameters_atomically`

Node parameters

Parameters–related callbacks:

- pre-set callback – e.g. add changes of dependent *parameter*s
- on-set callback – validate or rejects changes
- post-set callback – update internal state, react to change

`/parameter_events` – a *topic* for observation of *parameter*s changes of all *node*s

ROS Client Library (RCL)★ – APIs that allow users to implement their *ROS 2*

- gives access to *ROS 2* concepts such as *node*s, *topic*s, *service*s, etc.
- available for variety of programming languages, e.g.:
  - rclcpp: for C++
  - rclpy: for Python

Time in *ROS 2* is not just "the computer clock"★

ROS 2 has a time abstraction so the same code can run on:

- a real robot: normal wall/system time
- a simulator: simulated time
- rosbag playback: recorded time

3 types of time:

- system time – tied to the operating system wall clock
- steady time – monotonic, should not jump
- ROS time – used in *message*s and by *ROS 2* algorithms; it is:
  - the same as system time when no ROS time source is active
  - the latest value from ROS time source (`/clock`) when `use_sim_time` *parameter* is set

Executors

An *executor*★ is the object that decides when callbacks run and on which thread

A callback is a procedure called to handle a specific event, e.g.:

- subscription callbacks (receiving and handling data from a topic)
- timer callbacks
- service callbacks (for executing service requests in a server)
- different callbacks in action servers and clients
- parameters callbacks
- done-callbacks of Futures

Almost everything in *ROS 2* is a callback

- even a "synchronous" call to service / action adds a Future's done-callback

Main types of executors:

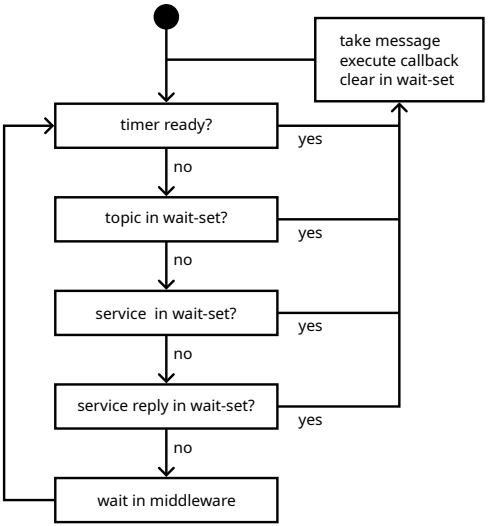
- SingleThreadedExecutor – executes all callbacks on a single thread, sequentially
- MultiThreadedExecutor – uses a thread pool; allows parallel execution of callbacks

Additionally:

- StaticSingleThreadedExecutor – a single thread executor for nodes with static interfaces; offers better performance
- WaitSet – allows to write a custom executor suited for a specific RT-demanding use case

SingleThreadedExecutor

- if processing time of callbacks $<$ period with which messages and events occur
  - ◇ incoming messages and events are processed in FIFO order
- if callbacks take too long
  - ◇ messages and events will be queued in the middleware
  - ◇ order of messages, services and events may be distorted
  - ◇ callbacks may be delayed



SingleThreadedExecutor

- good for:
  - simple nodes
  - low-rate logic
  - nodes with short callbacks
  - nodes where callbacks should not overlap
- bad for:
  - long service callbacks
  - blocking service calls inside callbacks
  - heavy perception callbacks
  - callbacks that wait for other callbacks to complete

MultiThreadedExecutor

- has several worker threads
- runs callbacks in a pool of threads
- callbacks may run in parallel, if callback groups allow it

Callback groups (CG):

- types:
  - Mutually Exclusive CG – callbacks in one group are executed as in SingleThreadedExecutor: only one callback can run at a time
  - Reentrant CG – parallel execution without restrictions, including different instances of the same callback
- callbacks belonging to different callback groups (of any type) can always be executed parallel to each other

Default CG: Mutually Exclusive CG

Callback Groups – use cases and examples:

- interaction of a callback with itself:
  - ◇ Reentrant CG, e.g.: a service server processing several calls in parallel to each other
  - ◇ Mutually Exclusive CG, e.g.: a timer callback that runs a control loop
- interaction of different callbacks with each other:
  - ◇ for mutual exclusion: use the same Mutually Exclusive CG, e.g.: callbacks are accessing shared critical and non-thread-safe resources
  - ◇ for parallel execution:
    - ○ two Mutually Exclusive CGs – no overlap of the individual callbacks
    - ○ a single Reentrant CG – overlap of the individual callbacks

When using shared resources and parallel execution:
remember to synchronize access

CG example: a deadlock (any executor)

```
  # default: MutuallyExclusiveCallbackGroup
  self.client = self.create_client(MySrv, 'my_service')
  self.timer = self.create_timer(0.1, self.timer_callback)

def timer_callback(self):
  request = MySrv.Request()
  future = self.client.call_async(request)
  rclpy.spin_until_future_complete(self, future)
  response = future.result()
```

CG example: delayed next timer callback (MultiThreadedExecutor)

```
  self.timer_cg = MutuallyExclusiveCallbackGroup()
  self.client_cg = MutuallyExclusiveCallbackGroup()

  self.client = self.create_client(MySrv, 'my_service'
                        callback_group=self.client_cg)

  self.timer = self.create_timer(0.1, self.timer_callback,
                        callback_group=self.timer_cg)

def timer_callback(self):
  request = MySrv.Request()
  future = self.client.call_async(request)
  rclpy.spin_until_future_complete(self, future)
  response = future.result()
```

CG example: asynchronous call (any executor)

```
def timer_callback(self):
  if self.pending_future is not None:
    return
  request = MySrv.Request()
  self.pending_future = self.client.call_async(request)
  self.pending_future.add_done_callback(self.resp_callback)

def resp_callback(self, future):
  self.pending_future = None
  try:
    response = future.result()
  except Exception as e:
    ...   # Handle error
    return
  # Use the response
```

Spinning the executor

- `spin()` – for long-running nodes
- `spin_once()` – in custom loops or tests
- `spin_until_future_complete()` – in startup; can be dangerous inside callbacks

A Future – represents a result that is not available yet

Callback-style:

```
  self.future = self.client.call_async(request)
  self.future.add_done_callback(self.done_callback)

def done_callback(self, future):
  ...
```

Polling-style:

```
  self.future = self.client.call_async(request)

def timer_callback(self):
  if self.future.done():
    response = self.future.result()
```

Blocking-style:

```
future = client.call_async(request)
rclpy.spin_until_future_complete(node, future)
response = future.result()
```

A Future can be in state:

- pending – result is not available yet
- done – result is available
- failed – exception/error occurred
- cancelled – future was cancelled

API:

- rclpy uses Python API★
- rclcpp uses `std::future<ServiceT::Response::SharedPtr>`

A *component*★ – a node implementation that can be loaded into a running process:

- no main() function
- subclass of rclcpp::Node; it is a kind of plugin
- fully supported in C++ only

Registration: macros from the package rclcpp_components, e.g.:

```
class MyComponent : public rclcpp::Node
{
  public:
    explicit MyComponent(const rclcpp::NodeOptions & opt)
      : rclcpp::Node("my_component", opt)
    { ... }
};
#include "rclcpp_components/register_node_macro.hpp"
RCLCPP_COMPONENTS_REGISTER_NODE(my_package::MyComponent)
```

Composition

The node/process layout is a deploy-time decision

Composition★ – pros and cons:

- multiple nodes in **separate processes** – process/fault isolation, easier debugging
- multiple nodes in a **single process** – lower overhead, more efficient communication (intra-process communication★)

`ros2 launch` – automates composition

Composition

Component container – types:

- component_container – a single SingleThreadedExecutor to execute all *component*s
- component_container_mt – a single MultiThreadedExecutor to execute the components
- component_container_isolated – a dedicated executor for each *component*: SingleThreadedExecutor (default) or MultiThreadedExecutor

Composing *component*s – methods:

- a generic container process + call to the ROS service load_node
- a custom executable containing multiple nodes which are known at compile time
- a *launch file* to create a container process with multiple *component*s loaded

Composition in launch.xml (what about callback groups?)

```
<launch>
    <node_container pkg="rclcpp_components"
                    exec="component_container_mt"
                    name="demo_container"  namespace=""
                    output="screen">
        <composable_node pkg="my_package"
                         plugin="my_package::NodeA"
                         name="talker"/>
        <composable_node pkg="my_package"
                         plugin="my_package::NodeB"
                         name="listener"/>
  </node_container>
</launch>
```

Composition in Python:

- a custom Python script running multiple nodes
- no *launch file* support
- executor defined in the script

```
import rclpy
from rclpy.executors import MultiThreadedExecutor
from my_package.nodes import NodeA, NodeB
def main():
    rclpy.init()
    node_a = NodeA()
    node_b = NodeB()
    executor = MultiThreadedExecutor()
    executor.add_node(node_a)
    executor.add_node(node_b)
    try:
        executor.spin()
    finally:
        executor.shutdown()
        node_a.destroy_node()
        node_b.destroy_node()
        rclpy.shutdown()
```

Lifecycle★

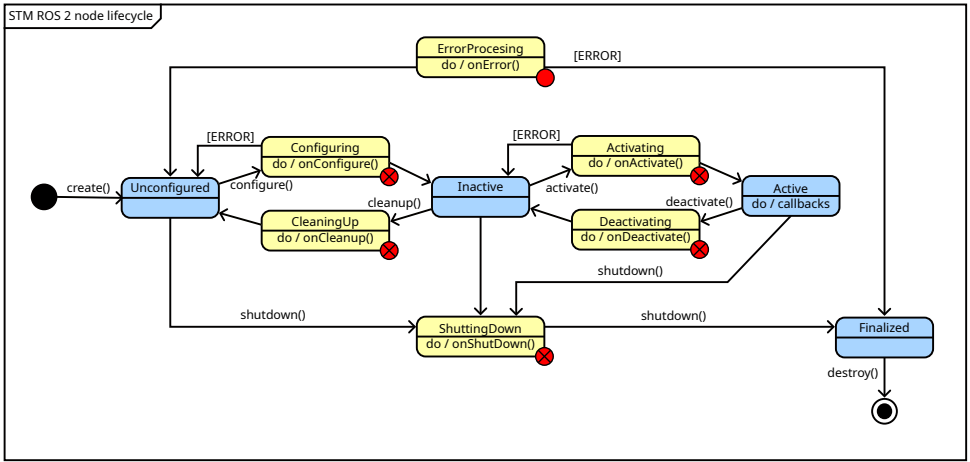
A *lifecycle node* – a *node* with a standardized state machine for startup, activation, deactivation, cleanup, shutdown, and error handling

- it is also called a *managed node*
- different than: "starting and doing everything immediately"

There are 4 primary states:

- unconfigured – *node* exists, but has not initialized its main resources yet
- inactive – *node* is configured and ready, but not actively processing/publishing
- active – *node* is running normally
- finalized – *node* has been shut down and should not be reused

+ 6 transition states



A *lifecycle-managed* *component*:

- is derived from rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface
- implements:
  - `on_configure`
  - on_cleanup
  - on_shutdown
  - on_activate
  - on_deactivate
  - on_error
  - destructor
- is a:
  - *lifecycle node*
  - *ros2_control*-related object:
    - *controller*
    - *hardware component*

Interface definitions

Interface Definition Language (IDL)★

- defines types (structures) for *message*, *service* and *action*
- enables automatic source-code generation

Interface definitions should be in dedicated *package*s, in directories:

- `msg/`: `.msg` files for *message*s
- `srv/`: `.srv` files for *service*s
- `action/`: `.action` files for *action*s

A `.msg` file consists of:

- **Fields** (typed data members)
- **Constants** (compile-time, unchangeable values)

Each field is defined as: `fieldtype fieldname`

```
int32 my_int
string my_string
```

Field names must:

- use lowercase alphanumeric characters + underscores
- start with a letter
- not end with an underscore
- not contain consecutive underscores

Field types can be:

- **Built-in types** (e.g., `int32`, `float64`, `string`)
- **Other messages** (e.g., `geometry_msgs/PoseStamped`)

Arrays and strings:

```
int32[] unbounded_integer_array
int32[5] five_integers_array
int32[<=5] up_to_five_integers_array

string string_of_unbounded_size
string<=10 up_to_ten_characters_string

string[<=5] up_to_five_unbounded_strings
string<=10[] unbounded_array_of_strings_up_to_ten_characters_each
string<=10[<=5] up_to_five_strings_up_to_ten_characters_each
```

Default values

Default values can be set for many primitive fields:

```
fieldtype fieldname fielddefaultvalue
```

- String defaults must be in single or double quotes
- Defaults are not supported for string arrays and complex (nested) types

```
uint8 x 25
int16 y -2000
string student_name "John"
int32[] samples [-200, -100, 0, 100, 200]
```

Constants

Constants (names must be UPPERCASE):

```
constanttype CONSTANTNAME=constantvalue
```

```
int32 X=123
string PLANET="Earth"
string STAR='Sun'
```

A `.srv` file contains:

- a request *message* (top)
- a separator: `---`
- a response *message* (bottom)

An example `.srv`:

```
string question
---
string answer
```

An `.action` file contains:

- a goal *message* (request)
- a separator: `---`
- a result *message* (response)
- a separator: `---`
- a feedback *message*

```
trajectory_msgs/JointTrajectory trajectory
---
int32 error_code
int32 SUCCESSFUL = 0
int32 INVALID_GOAL = -1
---
Header header
string[] joint_names
trajectory_msgs/JointTrajectoryPoint desired
trajectory_msgs/JointTrajectoryPoint actual
trajectory_msgs/JointTrajectoryPoint error
```

ROS Middleware Interface (*RMW*)★ – the abstraction layer between the *ROS 2* client libraries (rclcpp, rclpy) and the actual communication middleware underneath

A simplified stack:

- *node* code
- rclcpp / rclpy
- rcl
- *RMW*
- DDS / RTPS / Zenoh / other middleware
- network / shared memory / OS transport

RMW implementations:

- `rmw_fastrtps_cpp` (default)
- `rmw_cyclonedds_cpp`
- `rmw_connextdds`
- `rmw_gurumdds_cpp`
- `rmw_zenoh_cpp`

RMW must be the same for 2 *node*s to communicate

How to check the selected *RMW* implementation:

- `echo $RMW_IMPLEMENTATION` – if empty, default is used
- `ros2 doctor --report | grep -i rmw`

How to select *RMW*:

```
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
```

RMW is usually not visible, but may affect some properties, e.g.:

- discovery behavior
- multi-machine communication
- Quality of Service compatibility behavior
- latency
- throughput
- CPU usage
- memory usage
- large message performance
- Wi-Fi behavior
- reliability under packet loss

Quality of service (*QoS*)★ – the communication contract between endpoints

It controls:

- reliability
- buffering
- late-joining behavior
- timing expectations
- liveliness/failure detection

A connection can be:

- reliable as TCP
- best-effort as UDP
- + many levels between

QoS profile:

- history: `keep last` / `keep all`
  - `keep last`: keep only the last N samples
  - `keep all`: keep all samples, subject to middleware resource limits
- depth: the queue size used with `keep last`
- reliability: `reliable` / `best effort`
  - `reliable`: try to guarantee delivery, may retry, good for important data
  - `best_effort`: try to deliver, but samples may be lost, good for high–rate sensor data
- durability: `volatile` / `transient local`
  - `volatile`: old messages are not saved for future subscribers
  - `transient local`: publisher keeps samples for late subscribers
- deadline: maximum time between messages
- lifespan: how long a message remains valid
- liveliness, lease duration: how to detect, if publisher is alive

QoS compatibility

- *QoS* belongs to endpoints (publisher – subscriber, client – server)
- endpoints are configured independently
- connection is possible if the pair has compatible *QoS* profiles: offered *QoS* is less rigorous than requested *QoS*
  - subscriber – request *QoS*, "minimum quality" that it is willing to accept
  - publisher – offers *QoS*, "maximum quality" that it is able to provide

publisher	subscriber	compatible
`best effort`	`best effort`	Yes
`best effort`	`reliable`	No
`reliable`	`best effort`	Yes
`reliable`	`reliable`	Yes

Table: Compatibility of reliability

publisher	subscriber	compatible	result
`volatile`	`volatile`	Yes	New messages only
`volatile`	`transient local`	No	No communication
`transient local`	`volatile`	Yes	New messages only
`transient local`	`transient local`	Yes	New and old messages

Table: Compatibility of durability

Latched topic – both endpoints must agree to use `transient local`

Other compatibilities of *QoS* policies – in docs: deadline, liveliness, lease duration

QoS–related callbacks: deadline missed, liveliness changed, incompatible *QoS*

ROS 2 provides predefined *QoS* profiles, such as:

- default
- sensor data
- services
- parameters
- system default

pluginlib★ – the standard ROS mechanism for loading C++ classes at runtime from shared libraries

- *plugin*:
  - is of class derived from the **base interface**
  - is compiled into a shared library
  - is registered in *pluginlib* using a **type name** specified in a `xml` file
- *node*:
  - calls *pluginlib* to load an object of class of a specified **type name**
- *pluginlib*:
  - finds and loads the shared library
  - creates the requested object and returns a pointer
- *node*:
  - uses the object through a pointer with a defined **base interface**

Questions?